

Go-Zero IM 项目企业级升级完整计划书：V7-V15 后续升级路线

一、当前定位修正

前面 V1-V6 的升级目标主要是把项目打造成：

分布式 IM 消息投递内核

它重点解决：

可靠消息投递
多 Gateway 在线路由
ACK 与离线补偿
Kafka 群聊异步扩散
traceId 与压测
AI 产品事件通知预留

但图片中展示的“企业级 IM 项目”不只是消息投递内核，它还包括：

用户系统
API 网关
联系人系统
群组系统
会话系统
离线数据同步
文件与媒体消息
消息搜索
朋友圈 / 动态模块
系统通知
管理后台
监控告警
容器化部署
AI 智能体模块

所以完整升级路线应该分成四层：

L1：消息投递内核层：V1-V6
L2：企业级 IM 产品层：V7-V10
L3：生产工程化层：V11-V13
L4：AI 服务融合层：V14-V15

二、完整版本规划

V0: 现状梳理与基线整理

V1: 消息模型与单聊可靠投递

V2: 多 Gateway 在线路由与长连接治理

V3: ACK、pending、断线重连与离线补偿

V4: Kafka 群聊异步扩散与幂等消费

V5: 可观测性、压测与工程化基础

V6: 实时事件通知扩展，为 AI 产品复用预留能力

V7: 用户认证、账号体系与 API 网关

V8: 联系人、好友关系与群组管理

V9: 会话列表、未读数、已读回执与多端同步

V10: 离线数据同步、历史消息分页与消息搜索

V11: 文件 / 图片 / 语音等媒体消息与对象存储

V12: 朋友圈 / 动态模块与内容流系统

V13: 管理后台、风控、限流与安全治理

V14: Prometheus、Grafana、Sentry、日志检索与生产级可观测

V15: AI 智能体服务融合: RAG、Tool Calling、任务事件推送

其中:

V1-V6: 后端核心深度，必须优先完成

V7-V10: 企业级 IM 产品能力，秋招项目显著升级

V11-V13: 项目完整度和工程复杂度升级

V14: 生产环境工程化升级

V15: AI 产品融合升级，可作为未来 AI 产品底座

V7: 用户认证、账号体系与 API 网关

目标

把项目从“能连接 WebSocket”升级为具备完整用户入口的企业级系统。

这一版要解决:

用户如何注册 / 登录?

Token 如何签发和校验?

HTTP API 和 WebSocket 如何统一鉴权?

内部服务如何隔离?

是否有 API 网关层?

需要新增的模块

im-api-gateway
im-auth
im-user

核心功能

1. 用户注册与登录

支持：

手机号 / 邮箱 / 用户名注册
密码登录
JWT Token 签发
Refresh Token 可选
用户基础信息查询

2. JWT 鉴权

HTTP 请求：

客户端携带 Authorization: Bearer token
API Gateway 校验 Token
转发到后端服务

WebSocket 连接：

客户端连接时携带 token
Gateway 校验 token
解析 userId / deviceId
建立连接

3. API Gateway

API Gateway 负责：

统一鉴权
请求转发
限流入口
统一错误码
traceId 注入
接口日志

数据表

```
users
- id
- username
- phone
- email
- password_hash
- avatar
- status
- created_at
- updated_at

user_devices
- id
- user_id
- device_id
- device_type
- last_login_time
- created_at
```

验收标准

1. 用户可以注册、登录、获取 token
2. HTTP API 需要 token 才能访问
3. WebSocket 建连时必须校验 token
4. Gateway 能从 token 中解析 userId
5. 所有请求自动生成 traceId

简历价值

可以写：

设计用户认证与 API 网关层，基于 JWT 实现 HTTP API 与 WebSocket 长连接统一鉴权，并在网关注入 traceId、统一错误码与请求日志，提升系统入口层可维护性。

Codex 任务

请基于当前 Go-Zero IM 项目新增用户认证与 API Gateway 层。要求实现用户注册、登录、JWT Token 签发、HTTP API 鉴权、WebSocket 建连鉴权，并将 traceId 注入到后续请求链路。请优先保持现有消息链路稳定，不要大规模重构已有 Gateway 和 Logic。

V8：联系人、好友关系与群组管理

目标

把 IM 从“用户之间直接发消息”升级为真正的社交关系系统。

这一版要解决：

谁可以给谁发消息？
好友关系怎么建立？
群怎么创建？
群成员怎么管理？
群消息如何校验权限？

新增模块

im-contact
im-group

核心功能

1. 好友系统

支持：

搜索用户
发送好友申请
同意 / 拒绝好友申请
好友列表
删除好友
拉黑用户

2. 群组系统

支持：

创建群
邀请入群
退出群
群成员列表
群角色：群主 / 管理员 / 普通成员
踢人
解散群

3. 消息发送权限校验

单聊：

只有好友关系正常时可以发送
被拉黑时不可发送

群聊：

只有群成员可以发送群消息
被禁言用户不可发送
群解散后不可发送

数据表

friend_relations

- id
- user_id
- friend_id
- status
- created_at
- updated_at

friend_requests

- id
- from_user_id
- to_user_id
- message
- status
- created_at
- updated_at

groups

- id
- group_id
- name
- avatar
- owner_id
- status
- created_at
- updated_at

group_members

- id
- group_id
- user_id
- role
- mute_until

- joined_at
- status

验收标准

1. 非好友不能直接单聊，或需要根据策略控制
2. 用户可以申请好友、同意好友、删除好友
3. 用户可以创建群、邀请成员、退出群
4. 群消息发送前会校验群成员身份
5. 群成员变动后，群聊扩散能读取最新成员关系

简历价值

可以写：

实现联系人与群组管理模块，支持好友申请、好友关系维护、群创建、成员角色管理和消息发送权限校验，保证单聊与群聊链路在关系状态变化下的正确性。

Codex 任务

请在当前 IM 项目中新增联系人与群组管理能力，包括好友申请、好友列表、删除好友、群创建、群成员管理、群角色和消息发送权限校验。要求群聊消息在发送前校验发送者是否属于该群，并与现有 Kafka 群聊扩散链路兼容。

V9：会话列表、未读数、已读回执与多端同步

目标

把项目从“能发消息”升级为“完整 IM 客户端体验”。

这一版要解决：

用户打开 App 怎么看到会话列表？
每个会话最后一条消息是什么？
未读数怎么算？
多端登录时消息状态如何同步？
已读回执怎么处理？

核心功能

1. 会话列表

每个用户维护自己的会话列表：

```
conversation_id
对方用户 / 群组
最后一条消息
最后更新时间
置顶状态
免打扰状态
未读数
```

2. 未读数

消息投递给用户后：

```
如果用户当前未打开该会话，未读数 +1
如果用户打开会话并上报 readSeq，未读数清零或减少
```

3. 已读回执

客户端上报：

```
conversationId
readSeq
```

服务端记录用户已读到哪条消息。

4. 多端同步

同一用户多个设备在线时：

```
手机端读了消息
电脑端未读数也要更新
```

数据表

```
user_conversations
- id
- user_id
- conversation_id
- conversation_type
- target_id
```



```
- last_msg_id
- last_seq
- read_seq
- unread_count
- is_top
- is_muted
- updated_at

read_receipts
- id
- conversation_id
- user_id
- read_seq
- read_time
```

验收标准

1. 用户登录后能获取会话列表
2. 会话列表按最后消息时间排序
3. 新消息到达后对应会话未读数增加
4. 用户上报 readSeq 后未读数更新
5. 多端在线时，已读状态能同步到其他设备

简历价值

可以写：

设计用户会话列表与未读数模型，基于 readSeq 维护用户在各会话中的已读位置，并支持多端在线状态下的已读回执同步，提升 IM 客户端一致性体验。

Codex 任务

请在当前 IM 项目中实现会话列表、未读数和已读回执能力。要求设计 user_conversations 表，支持获取会话列表、新消息更新最后一条消息和未读数、客户端上报 readSeq、服务端同步已读状态到同一用户其他在线设备。

V10：离线数据同步、历史消息分页与消息搜索

目标

把项目升级为具备完整历史数据能力的企业级 IM。

这一版要解决：

用户很久没登录，如何同步离线期间的数据？
历史消息怎么分页？
消息太多怎么搜索？
群聊历史怎么拉取？

核心功能

1. 历史消息分页

支持：

按 conversationId 查询
按 seq 范围查询
按时间倒序分页
游标分页

推荐接口：

```
GET /messages?conversationId=xxx&beforeSeq=100&limit=20
```

2. 离线数据同步

用户重连后不仅补消息，还要同步：

会话变更
群成员变更
好友关系变更
系统通知
未读数变化

3. 消息搜索

初期可以先做 MySQL LIKE 或全文索引。

高级版接入 Elasticsearch：

message 写入 MySQL 后异步写入 ES
用户搜索关键词时查询 ES
返回 messageId 后回查 MySQL

数据设计

```
sync_events
- id
- user_id
- event_type
- event_id
- seq
- payload
- created_at
```

事件类型：

```
message_created
conversation_updated
friend_changed
group_member_changed
system_notice
```

验收标准

1. 用户可以分页拉取历史消息
2. 用户断线后重连可以同步离线期间的消息和会话变化
3. 支持按关键词搜索消息
4. 搜索结果可以定位到具体会话和消息
5. 数据同步不会重复应用同一个事件

简历价值

可以写：

设计基于 sync_events 的离线数据同步模型，将消息、会话、好友和群成员变更统一抽象为用户级增量事件，支持客户端重连后按同步位点拉取离线变更，降低多端数据不一致问题。

Codex 任务

请为当前 IM 项目设计并实现离线数据同步与历史消息分页能力。要求支持按 conversationId 和 seq 分页查询历史消息，并新增 sync_events 表，将消息、会话、好友、群成员等变化抽象为用户级同步事件，客户端可根据 syncSeq 拉取增量数据。

V11：文件 / 图片 / 语音等媒体消息与对象存储

目标

让项目从纯文本 IM 升级为完整消息类型系统。

这一版要解决：

如何发送图片？
如何发送文件？
文件怎么上传？
消息内容如何扩展？
大文件是否经过 IM 服务转发？

核心设计

1. 对象存储

使用 MinIO 或云对象存储。

上传流程：

客户端申请上传凭证
服务端生成 uploadUrl
客户端直传对象存储
上传完成后发送媒体消息
消息中只保存 fileKey / url / metadata

2. 消息类型扩展

```
content_type:
1 = text
2 = image
3 = file
4 = voice
5 = video
6 = system_notice
7 = task_event
```

3. 文件元数据

```
files
- id
- file_key
- file_name
```

- file_size
- mime_type
- uploader_id
- storage_type
- created_at

验收标准

1. 支持申请文件上传凭证
2. 支持图片 / 文件消息发送
3. 消息表不直接存储大文件内容
4. 文件消息可以在客户端展示文件名、大小和下载地址
5. 权限校验：非会话成员不可访问文件

简历价值

可以写：

接入对象存储实现图片与文件消息，采用预签名上传降低 IM 服务带宽压力，并通过文件元数据表和会话权限校验控制媒体资源访问。

Codex 任务

请在当前 IM 项目中增加文件与图片消息能力。要求接入 MinIO 或本地对象存储模拟，支持申请上传凭证、保存文件元数据、发送 image/file 类型消息，并确保消息表只保存 fileKey 和 metadata，不直接存储文件内容。

V12：朋友圈 / 动态模块与内容流系统

目标

达到图片中“朋友圈模块 / Moment 模块”的项目完整度。

这一版不是 IM 核心，但能让项目看起来更接近完整社交产品。

核心功能

- 发布动态
- 上传图片
- 好友可见
- 点赞
- 评论

删除动态
动态列表

数据表

```
moments
- id
- user_id
- content
- media_list
- visibility
- created_at
- deleted_at

moment_likes
- id
- moment_id
- user_id
- created_at

moment_comments
- id
- moment_id
- user_id
- content
- created_at
```

权限逻辑

只有好友能看到动态
被拉黑用户不可见
删除好友后可见性根据策略处理

验收标准

1. 用户可以发布动态
2. 好友可以看到动态
3. 支持点赞和评论
4. 支持图片动态
5. 删除动态后不可见

简历价值

这一版不是主亮点，不建议放在简历第一优先级。可以作为项目完整性描述。

可以写：

扩展 Moment 动态模块，支持好友可见范围下的动态发布、评论与点赞，复用联系人关系和对象存储能力完善社交产品闭环。

Codex 任务

请为当前 IM 项目新增 Moment 动态模块，支持发布动态、好友可见、点赞、评论和删除动态。要求复用已有好友关系进行可见性判断，并支持图片动态。

V13：管理后台、风控、限流与安全治理

目标

把项目从“用户侧功能完整”升级到“企业级后台可管理”。

这一版要解决：

异常用户怎么封禁？
垃圾消息怎么限制？
接口被刷怎么办？
群聊刷屏怎么办？
管理员如何查看系统数据？

核心功能

1. 管理后台 API

支持：

用户查询
用户封禁 / 解封
群组查询
消息查询
敏感词管理
系统公告

2. 限流

可以使用 Redis 实现：

用户发送消息频率限制
登录接口限制
好友申请频率限制
群聊发言频率限制

3. 敏感词过滤

文本消息发送前：

检测敏感词
命中后拒绝发送或替换
记录风控日志

4. 黑名单与封禁

用户被封禁后不可登录 / 不可发消息
群成员被禁言后不可发群消息

数据表

admin_users
risk_logs
sensitive_words
user_bans
rate_limit_rules

验收标准

1. 管理员可以查询用户和消息
2. 可以封禁用户
3. 被封禁用户不可发送消息
4. 消息发送有频率限制
5. 敏感词命中后有处理逻辑
6. 风控操作有日志记录

简历价值

可以写：

补充后台治理能力，基于 Redis 实现消息发送、登录和好友申请等高频接口限流，并引入用户封禁、群禁言和敏感词过滤机制，提升系统安全性和抗滥用能力。

Codex 任务

请为当前 IM 项目新增基础后台治理能力，包括管理员接口、用户封禁、群禁言、敏感词过滤和基于 Redis 的接口限流。要求封禁和禁言状态能影响消息发送链路，并记录风控日志。

V14: Prometheus、Grafana、Sentry、日志检索与生产级可观测

目标

达到图片中“Prometheus、Grafana、Sentry、Docker”等工程化展示水平。

这一版要解决：

系统运行状态如何观测？
错误如何收集？
指标如何展示？
服务挂了如何发现？
日志如何检索？

核心能力

1. Prometheus 指标

暴露指标：

```
im_gateway_connections
im_message_send_total
im_message_send_fail_total
im_message_ack_total
im_message_latency_ms
im_kafka_consumer_lag
im_online_users
im_reconnect_total
```

2. Grafana Dashboard

展示：

在线连接数
消息 QPS
消息成功率

P95 延迟
Gateway CPU / 内存
Kafka 消费积压
重连次数
ACK 成功率

3. Sentry 错误收集

收集：

panic
接口异常
Kafka 消费异常
数据库异常
Redis 异常

4. 日志检索

可选方案：

ELK
Loki + Grafana
本地结构化日志 + grep

学生项目可先做轻量版：

结构化日志 + traceId + 可检索日志文件

Docker Compose

需要包含：

MySQL
Redis
Kafka
Etcd
Gateway
Logic
Transfer
Prometheus
Grafana
MinIO
可选 Elasticsearch

验收标准

1. Prometheus 能抓取服务指标
2. Grafana 能看到核心 Dashboard
3. 服务异常有日志和错误记录
4. Docker Compose 一键启动主要依赖
5. 压测时 Dashboard 能看到连接数和消息 QPS 变化

简历价值

可以写：

接入 Prometheus 与 Grafana 构建 IM 系统监控面板，覆盖在线连接数、消息 QPS、投递成功率、ACK 成功率和 P95 延迟等核心指标，并结合 traceId 结构化日志提升故障定位效率。

Codex 任务

请为当前 IM 项目接入 Prometheus 指标暴露，并提供 Grafana Dashboard 配置示例。要求至少统计在线连接数、消息发送总数、失败数、ACK 数量、消息延迟和 Kafka 消费积压。同时补充 Docker Compose，使 MySQL、Redis、Kafka、Etcd、Prometheus、Grafana 和核心服务可以一键启动。

V15：AI 智能体服务融合：RAG、Tool Calling、任务事件推送

目标

达到图片中“AI 智能体模块 / Eino / RAG / MCP / Tool Calling”的方向，但不要强行污染 IM 主链路。

正确做法是：

IM 系统负责实时消息与事件通知
AI Service 作为独立服务接入
AI 任务状态通过 IM 事件通道推送

新增模块

im-ai-service
im-task-service

核心功能

1. AI 会话

用户可以创建 AI 会话：

用户发送问题
AI Service 调用大模型
流式返回结果
结果通过 WebSocket 推送

2. RAG 检索

支持：

上传文档
文档切分
向量化
检索相关片段
拼接上下文
生成回答

学生项目可以简化：

先用本地知识库 + pgvector / Milvus / 简单向量库

3. Tool Calling

支持工具：

查询用户资料
查询会话历史
查询任务状态
生成总结
生成待办

4. AI 任务事件推送

复用 V6 的事件消息：

TaskCreated
TaskRunning
AgentThinking
ToolCalling
RagSearching

```
AnswerStreaming
TaskFinished
TaskFailed
```

通过 WebSocket 推给客户端。

消息类型

```
message_type:
text
system_notice
task_event
agent_event
ai_stream
```

验收标准

1. 用户可以创建 AI 会话
2. AI 回复可以通过 WebSocket 流式返回
3. AI 任务状态通过 task_event 推送
4. RAG 检索可以返回相关上下文
5. AI Service 与 IM 主链路解耦
6. AI 失败不会影响普通聊天消息

简历价值

这一版可以作为第二亮点，而不是覆盖 IM 主项目。

可以写：

在 IM 实时消息底座上扩展 AI Service，支持 AI 会话、RAG 检索和任务状态事件推送，将 Agent 执行过程抽象为 task_event / agent_event，通过 WebSocket 实时同步给前端，实现 IM 系统向 AI 任务通知底座的扩展。

Codex 任务

请在当前 IM 项目中新增独立 AI Service，不要破坏现有 IM 消息链路。AI Service 支持创建 AI 会话、调用大模型接口的抽象层、RAG 检索接口占位、Tool Calling 接口占位，并将 AI 执行状态通过 task_event / agent_event 消息类型推送到 WebSocket 客户端。

三、达到图片级项目的推荐优先级

如果目标是秋招简历，不需要全部完成 V7-V15。

最推荐完成：

必须完成：

- V1: 可靠消息
- V2: 多 Gateway 在线路由
- V3: ACK 与离线补偿
- V4: Kafka 群聊扩散
- V5: 压测与 traceId
- V7: 用户认证与 API Gateway
- V8: 好友与群组
- V9: 会话列表与未读数

加分完成：

- V10: 历史消息分页与离线数据同步
- V11: 文件 / 图片消息
- V14: Prometheus + Grafana + Docker Compose

可选完成：

- V12: 朋友圈
- V13: 管理后台与风控
- V15: AI Service

如果要达到图片中那种“企业级 IM 项目介绍页”的展示完整度，至少需要：

V1-V11 + V14

如果要达到“IM + AI 智能体模块”的展示完整度，需要：

V1-V11 + V14 + V15

朋友圈、红包这类功能不是后端秋招核心，可以不做。

四、最终项目形态

完成 V1-V11 + V14 后，项目可以描述为：

基于 Go-Zero 的企业级分布式 IM 系统，系统采用微服务架构，拆分 API Gateway、Auth、User、Gateway、Logic、Transfer、Contact、Group、Conversation、Message、OfflineSync、File 等服务，支

持用户认证、好友关系、群组管理、WebSocket 长连接、单聊消息可靠投递、群聊异步扩散、离线消息补偿、会话列表、未读数、历史消息分页、图片文件消息和生产级监控部署。

完成 V15 后，可以进一步描述为：

在 IM 实时通信底座上扩展 AI Service，将 AI 会话、RAG 检索和 Agent 执行过程抽象为实时任务事件，通过 WebSocket 向用户推送任务进度和流式结果，使系统可复用为 AI 产品的实时事件通知中心。

五、最终服务模块结构

建议项目最终结构：

```
cmd/  
  im-api-gateway/  
  im-auth/  
  im-user/  
  im-gateway/  
  im-logic/  
  im-transfer/  
  im-contact/  
  im-group/  
  im-conversation/  
  im-message/  
  im-offline-sync/  
  im-file/  
  im-admin/  
  im-ai-service/  
  
internal/  
  common/  
  protocol/  
  middleware/  
  auth/  
  redis/  
  kafka/  
  mysql/  
  trace/  
  metrics/  
  
docs/  
  architecture.md  
  upgrade_assessment.md  
  message_reliability.md  
  gateway_routing.md  
  offline_sync.md  
  kafka_group_delivery.md
```

```
benchmark_report.md  
deploy.md  
ai_event_integration.md
```

```
deploy/  
  docker-compose.yml  
  prometheus.yml  
  grafana-dashboard.json
```

六、最终数据表总览

核心表：

```
users  
user_devices  
  
friend_relations  
friend_requests  
  
groups  
group_members  
  
conversations  
user_conversations  
  
messages  
message_delivery  
read_receipts  
  
sync_events  
  
files  
  
moments  
moment_likes  
moment_comments  
  
admin_users  
risk_logs  
sensitive_words  
user_bans  
rate_limit_rules  
  
ai_sessions  
ai_messages
```



```
ai_tasks
ai_task_events
```

其中秋招重点表：

```
messages
message_delivery
user_conversations
sync_events
group_members
friend_relations
```

七、最终简历版本

项目名称

基于 Go-Zero 的企业级分布式 IM 与实时事件通知系统

技术栈

Go、Go-Zero、WebSocket、gRPC、Etcd、Redis、Kafka、MySQL、Docker、Prometheus、Grafana、MinIO

项目描述

面向高并发长连接和实时消息投递场景，设计并实现企业级分布式 IM 后端系统。系统采用微服务架构，拆分 API Gateway、Auth、User、Gateway、Logic、Transfer、Contact、Group、Conversation、Message、File 等核心服务，支持用户认证、好友关系、群组管理、WebSocket 长连接、单聊可靠投递、群聊异步扩散、离线消息补偿、会话列表、未读数、历史消息分页、文件消息和生产级监控部署，并可扩展为 AI 产品的实时任务事件通知底座。

项目亮点

1. 拆分 Gateway、Logic、Transfer 等核心服务，Gateway 负责 WebSocket 长连接维护、心跳检测和连接注册，Logic 负责消息路由、会话状态和业务处理，Transfer 负责 Kafka 异步消费和群聊扩散，实现连接层、业务层和异步投递层解耦。
2. 基于 Redis 维护 userId 到 Gateway 实例的在线路由关系，通过 TTL + 心跳续期处理异常断连，Logic 查询在线状态后通过 gRPC 调用目标 Gateway 完成实时投递，支持多 Gateway 横向扩展。
3. 设计 clientMsgId、serverMsgId、conversationSeq 和 message_delivery 投递表，结合 ACK、pending 状态和断线重连补偿机制，解决客户端重试、网络抖动和连接断开导致的消息重复与消息丢失问题。

题。

4. 针对群聊同步扩散链路过长的问题，引入 Kafka 构建异步扩散流程，Producer 以 conversationId 作为 key 保证单会话内顺序，Transfer 消费后批量生成投递任务，并通过唯一约束保证重复消费幂等。

5. 设计 user_conversations、readSeq 和 sync_events 模型，支持会话列表、未读数、已读回执、多端同步和离线增量数据同步，提升 IM 客户端数据一致性。

6. 接入 MinIO 实现图片与文件消息，采用预签名上传降低 IM 服务带宽压力，并通过文件元数据表和会话权限校验控制媒体资源访问。

7. 接入 Prometheus 与 Grafana 构建监控面板，覆盖在线连接数、消息 QPS、ACK 成功率、Kafka 消费积压和 P95 延迟等指标，并结合 traceId 结构化日志提升问题定位效率。

8. 预留 task_event、agent_event 等事件消息类型，可将 AI 任务创建、Agent 执行进度、Tool Calling 结果和任务完成状态通过 WebSocket 实时推送，支持后续复用为 AI 产品事件通知底座。

八、给 Codex 的总提示词

请基于当前 Go-Zero IM 项目，评估将其升级为“企业级分布式 IM 与实时事件通知系统”的完整可行性。当前 V1-V6 主要覆盖可靠消息、多 Gateway 路由、ACK、离线补偿、Kafka 群聊扩散、traceId、压测和 task_event 预留。现在请继续评估 V7-V15 的后续升级路线。

后续路线：

V7：用户认证、账号体系与 API Gateway

V8：联系人、好友关系与群组管理

V9：会话列表、未读数、已读回执与多端同步

V10：离线数据同步、历史消息分页与消息搜索

V11：文件 / 图片 / 语音等媒体消息与对象存储

V12：朋友圈 / 动态模块与内容流系统

V13：管理后台、风控、限流与安全治理

V14：Prometheus、Grafana、Sentry、日志检索与生产级可观测

V15：AI 智能体服务融合：RAG、Tool Calling、任务事件推送

请输出：

1. 当前项目距离企业级 IM 项目的差距
2. V7-V15 每个版本需要新增的服务模块
3. 每个版本涉及的数据表
4. 每个版本需要修改的现有模块
5. 每个版本的实现难度和风险
6. 每个版本的最小可落地版本
7. 哪些版本适合秋招前完成
8. 哪些版本可以秋招后继续做
9. 最终推荐的项目目录结构
10. 最终可以写进简历的项目描述和亮点

九、最现实的执行建议

秋招前必须优先

V1: 可靠消息
V2: 多 Gateway 路由
V3: ACK 与离线补偿
V4: Kafka 群聊扩散
V5: traceId、压测
V7: 认证与 API Gateway
V8: 好友与群组
V9: 会话列表、未读数

这几版完成后，项目已经明显强于普通 IM Demo。

秋招前有时间再做

V10: 离线数据同步
V11: 文件 / 图片消息
V14: Prometheus + Grafana + Docker Compose

这几版能把项目包装成比较完整的企业级系统。

秋招后再做

V12: 朋友圈
V13: 管理后台风控
V15: AI Service

这几版适合作为长期项目演进，不建议现在挤占实习和秋招时间。

十、最终判断

前面 V1-V6 解决的是：

IM 系统的消息投递深度

后面 V7-V15 解决的是：

企业级 IM 项目的产品完整度、工程完整度和 AI 扩展能力

如果目标只是秋招主项目，完成：

V1-V9 + V14 部分能力

就已经足够有竞争力。

如果目标是接近图片中的“企业级 IM + AI 智能体模块”完整项目，需要继续完成：

V1-V11 + V14 + V15

朋友圈、红包、复杂前端不是核心，不建议作为主线。